ASSIGNMENT

1 Bounded Buffer (Producer - Consumer)

Producer consumer problem is a classic synchronization problem where multiple process or threads share common buffer. Proper synchronization prevents race condition and ensure correct access to the shared resource.

- Producer inserts the data into the shared buffer
- Consumer removes and process data item from the shared buffer.

Synchronization requirements

Synchronization mechanism must satisfy the following conditions.

- Producer cannot insert item when the buffer is ~~empty~~ full.
- Consumer cannot remove item when the buffer is ~~full~~ empty.
- Only one process can access the shared buffer (critical section) at a time.

Semaphore : The synchronization tool

A semaphore is a integer based synchronization mechanism used to coordinate access to shared resource.

The semaphore solution use three semaphore

Semophor	Purpose
mutex	Provides mutual exclusion while accing shared buffer
full	Counts the number of buffer slots
empty	Counts the number of empty buffer slots

Semophore opration

2 atomic opration

wait(): Decrease the semophore count. If value becomes zero or negative the calling process wait until resource become available.

Signal(): Increase the semophore value & wake the waiting process if it exist

Semophore intialization

```
mutex = 1    // binary semophore for mutual exclusion
full = 0     // initially no slot filled
empty = n    // buffer size
```

Producer Algorithm

```
// check empty slot
// If yes enter critical section
// insert item & update semophr
do {
    // Produce an item
    wait(empty);
    wait(mutex);
    // insert item into buffer
    signal(mutex);
    signal(full);
} while(true);
```

Consumer Algorithm

```
// wait till buffer contains one
// enter critical section
// remove item & update
do {
    wait(full);
    wait(mutex);
    // Remove item from buffer
    signal(mutex);
    signal(full);
} while(true);
```

Free RTOS code

```
#define N 8
static uint8_t buf[N];
static int head=0, tail=0;
SemaphoreHandle_t empty, full, mutex;
void init(void){
    empty = xSemaphoreCreateCounting(N,N); // N free slot
    full = xSemaphoreCreateCounting(N,0); // 0 filled slot
    mutex = xSemaphoreCreateMutex();
}
/* Producer */
void Producer(void *p){
    for(;;){
        uint8_t item = produce_item();
        xSemaphoreTake(empty, portMAX_DELAY);
        xSemaphoreTake(mutex, portMAX_DELAY);
        buf[head] = item;
        head = (head+1) % N;
        xSemaphoreGive(mutex);
        xSemaphoreGive(full);
    }
}
/* Consumer */
void Consumer(void *p){
    for(;;){
        xSemaphoreTake(full, portMAX_DELAY);
        xSemaphoreTake(mutex, portMAX_DELAY);
        uint8_t item = buf[tail];
        tail = (tail+1) % N;
        xSemaphoreGive(mutex);
        xSemaphoreGive(empty);
        consume_item(item);
    }
}
```

2 Reader Writer problem

Reader writer problem is a classic synchronization problem in operating system. It deals with coordinating access to shared resource (database) by multiple process or thread.

- Reader: Multiple reader can read the shared data simultaneously without causing inconsistency
- Writers: Only one writer can access data at a time and no reader or another while writing (to prevent data corruption

Variants of the problem

1. Reader preference
   - reader are given priority
   - No reader wait for the resource if available for reading
   - Writers may suffer from starvation.

2. Writers preference
   - Writers are prioritized over reading. ~~even if~~
   - Ensure writer don't starve. but reader may wait longer.

Solution when reader has the priority over writer. Here priority means 'no reader should wait if the resource is currently open for reading.

Case	Process 1	Process 2	Allowed / Not Allowed
1	Writing	Writing	Not Allowed
2	Writing	Reading	Not Allowed
3	Reading	Writing	Not Allowed
4	Reading	Reading	Allowed

Synchronization variable

mutex: Ensures mutual exclusion when updating reader count (readcnt)

wrt: controls access to shared data

readcnt: No. of readers currently in critical section.

Semaphore Operation

wait(): Decrease semaphore value, block if <0.

signal(): Increase semaphore value, wake waiting process.

Reader Process (Reader preference)

1. Reader request the entry to critical section.
2. If allowed
   - Increase count of readers in critical section.
   - If first reader block wrt semaphore
   - It signals any other readers to read if already reading
   - After reading, exits critical section. Checks if other readers inside & info wrt to enter critical sec.
3. If not allowed keep waiting

Writer Process

- wait(wrt): Locks the resource to get exclusive use
- Critical section: Writer perform writing
- Signal(wrt): writer release resource after writing

- Reader preference solution allows multiple readers at a time to access simultaneously
- However it might cause starvation
- To avoid this Writer preference or Fair solution should be used to ensure there is no starvation.

```
static int read_count = 0
SemaphoreHandle_t rc_mutex;
SemaphoreHandle_t rw_mutex;
void rw_init(){
  rc_mutex = xSemaphoreCreateMutex();
  rw_mutex = xSemaphoreCreateBinary();
  xSemaphoreGive(rw_mutex);
}
void Reader(void *p){
  for(;;){
    xTake xSemaphoreTake(rc_mutex, portMAX_DELAY);
    read_count++;
    if(read_count == 1)
      xSemaphoreTake(rw_mutex, portMAX_DELAY);
    xSemaphoreGive(rc_mutex);
    read_data();
    xSemaphoreTake(rc_mutex, portMAX_DELAY);
    read_count--;
    if(read_count == 0)
      xSemaphoreGive(rw_mutex);
    xSemaphoreGive(rc_mutex);
    vTaskDelay(pdMS_TO_TICKS(10));
  }
}
void Writer(void *p){
  for(;;){
    xSemaphoreTake(rw_mutex, portMAX_DELAY);
    write_data();
    xSemaphoreGive(rw_mutex);
    vTaskDelay(pdMS_TO_TICKS(100));
  }
}
```

3 Dining Philosophers

The dining philosophers problem is a classic synchronization problem introduced by Edsger Dijstra in 1965. It illustrates the challenges of resource sharing in concurrent programming such as deadlock, starvation & mutual exclusion.

Problem

- K philosophers sit around circular table
- Each philosopher alternate between thinking and eating
- There is one chopstick between each philosopher (total K chopsticks)
- A philosopher must pick up 2 chopstick (left & write) to eat.
- Only one philosopher can use chopstick at a time.

The challenge: Design a synchronization mechanism so that philosopher can eat without causing deadlock (all waiting forever) or starvation (some never get a chance to eat)

Issues in the problem

1. Deadlock: If every philosopher picks up their left chopstick first, no one can pick up the right one creating wait
2. Starvation: Some philosopher never get a chance to eat if others keep eating
3. Concurrency control: Must ensure no two adjacent philosophers eat simultaneously.

Semaphore solution to Dining philosophe

We use semphore to manage chopstick & avoid deadlock.

Algorithm

- Each chopstick is present in binary semaphore (mutex)
- Philosopher must acquire both left and right Semaphore before eating
- After eating, the philosopher release both semaphore.

Pseudo code

```
semaphore chopstick [5] = { 1, 1, 1, 1, 1};

Philosopher (i) :
    while (true) {
        think();
        wait (chopstick [i]);
        wait (chopstick [(i+1)%5]);
        eat();
        signal (chopstick [(i+1)%5]);
        signal (chopstick [i]);
}
```

- Each chopstick binary semaphore is enabled to 1
- think() : philosopher thinking
- pick up left chopstick, if free
- pick up right chopstick, if free
- eat
- signal both left & right chopstick down.

```
#define N 5
SemaphoreHandle_t fork[N]

void phil_init(void){
    for(int i=0; i<N; i++)
        fork[i] = xSemaphoreCreateMutex();
}

void philosopher(void *arg){
    int id = (int)(intptr_t)arg;
    int left = id;
    int right = (id+1) % N;

    for(;;){
        // Think
        vTaskDelay(pdMS_TO_TICKS(200+(id*37)));

        // Pick up P4 first in order to break circular wait
        if(id == (N-1)){
            xSemaphoreTake(fork[right], portMAX_DELAY);
            xSemaphoreTake(fork[left], portMAX_DELAY);
        } else {
            xSemaphoreTake(fork[left], portMAX_DELAY);
            xSemaphoreTake(fork[right], portMAX_DELAY);
        }

        vTaskDelay(pdMS_TO_TICKS(100));

        xSemaphoreGive(fork[left]);
        xSemaphoreGive(fork[right]);
    }
}
```

4 Sleeping Barber

The sleping barber problem is a classic process synchronization problem that demonstrates how to manage concurrent process. It models a barber shop scenario with barber chair and a waiting room. Only one customer occupies a barber chair at a time, barber works whenever a customer is present and sleeps when no customer are waiting

Problem statement

1. One barber, one ~~chair~~ barber chair, n waiting chairs
2. If no customer barber sleeps in his chair
3. A customer who arrives
   - wakes up barber if he is sleeping
   - waits if there are free chairs
   - leaves if no chair are available

Challenge

- Only one customer is in the barber chair at a time
- Customer either wait or leave when seats are full.
- The barber works when customer are available and sleep otherwise.

Synchronization solution

We are three semaphores

1. Customer count waiting customers.
2. Barber signals when barber is ready
3. Mutex ensure mutual exclusion when accessing chairs.

Algorithm

- Customer:
  - Tries to sit (if free seat > 0)
  - If seated increments waiting count & signals the barber
  - If no seat leave
- Barber
  - Sleeps if no customer
  - Wake up when signaled
  - Serves one customer at a time

Explanation

- Customer count waiting customers. Barber signals availability. Seats (mutex) protects chair count and Free seats = N tracks empty chairs.
- Barber loop: wait (down (customer)) until a barber arrives. Lock seats. free one chair (free seat ++).
  Signals the barber is ready (up(Barber)). unlock seats and cuts hair.
- Customer loop: locks seats. if a chair is free (FreeSeats > 0). takes it (FreeSeats --). notifies barber (up(customer)) unlock seats then waits for the barber (down(Barber)) to get haircut. If no seat is free, unlock seats and leaves

```
#define N_Seats 4
static int waiting = 0;

SemaphoreHandle_t customers;
SemaphoreHandle_t barber;
SemaphoreHandle_t sb_mutex;
```

```
void sb_init(void){
    customers = xSemaphoreCreateCounting(N_SEATS, 0);
    barber = xSemaphoreCreateBinary();
    sb_mutex = xSemaphoreCreateMutex();
}

/* Barber task */
void BarberTask(void *p){
    for(;;){
        xSemaphoreTake(customers, portMAX_DELAY);
        xSemaphoreTake(sb_mutex, portMAX_DELAY);
        waiting--;
        xSemaphoreGive(barber);
        xSemaphoreGive(sb_mutex);
        cut_hair();
    }
}

/* Customer task (many instances / ISR spawned) */
void CustomerTask(void *p){
    xSemaphoreTake(sb_mutex, portMAX_DELAY);
    if(waiting < N_SEATS){
        waiting++;
        xSemaphoreGive(customers);
        xSemaphoreGive(sb_mutex);
        xSemaphoreTake(barber, portMAX_DELAY);
        getHairCut()
    }
    else{
        xSemaphoreGive(sb_mutex);
    }
    vTaskDelete(NULL);
}
```